Name: VISHAL K

Reg no:212225060301

Ex.No.6 Development of Python Code Compatible with Multiple AI Tools

Aim:

Write and implement Python code that integrates with multiple AI tools to automate the task of interacting with APIs, comparing outputs, and generating actionable insights with Multiple AI Tools.

Explanation:

Develop a python code that integrates multiple AI tool by interacting with their APIs.
Compare outputs from different APIs.
Analyze the response and the Output.

The aim is to understand how to request help from AI tools for tasks like writing Python code, integrating with APIs, comparing outputs, and generating actionable insights.

Positive outcome:

```python
from nltk.sentiment import SentimentIntensityAnalyzer
import nltk

nltk.download('vader_lexicon')

# Simulated AI-generated text
generated_text = "This smartphone offers outstanding battery life and an intelligent AI camera that captures stunning photos.'

print("Generated Review:\n")
print(generated_text)

# Sentiment analysis
sia = SentimentIntensityAnalyzer()
sentiment = sia.polarity_scores(generated_text)

print("\nSentiment Analysis:")
print(sentiment)

# Insight generation
if sentiment['compound'] > 0:
    print("\nInsight: The review is positive and suitable for marketing promotion.")
else:
    print("\nInsight: The review tone is neutral or negative.")
```

[nltk_data] Downloading package vader_lexicon to /root/nltk_data... Generated Review:

This smartphone offers outstanding battery life and an intelligent AI camera that captures stunning photos.

Sentiment Analysis:
{'neg': 0.0, 'neu': 0.556, 'pos': 0.444, 'compound': 0.8625}

Insight: The review is positive and suitable for marketing promotion.

Negative outcome:

```python
from nltk.sentiment import SentimentIntensityAnalyzer
import nltk

nltk.download('vader_lexicon')

# Simulated AI-generated text
generated_text = "This smartphone offers poor battery life and an unreliable AI camera that captures disappointing photos."

print("Generated Review:\n")
print(generated_text)

# Sentiment analysis
sia = SentimentIntensityAnalyzer()
sentiment = sia.polarity_scores(generated_text)

print("\nSentiment Analysis:")
print(sentiment)

# Insight generation
if sentiment['compound'] > 0:
    print("\nInsight: The review is positive and suitable for marketing promotion.")
else:
    print("\nInsight: The review tone is neutral or negative.")
```

Generated Review:

This smartphone offers poor battery life and an unreliable AI camera that captures disappointing photos.

Sentiment Analysis:
{'neg': 0.326, 'neu': 0.674, 'pos': 0.0, 'compound': -0.743}

Insight: The review tone is neutral or negative.
[nltk_data] Downloading package vader_lexicon to /root/nltk_data...
[nltk_data] Package vader_lexicon is already up-to-date!

Neutral outcome:

```
from nltk.sentiment import SentimentIntensityAnalyzer
import nltk

nltk.download('vader_lexicon')

# Simulated AI-generated text
generated_text = "This poor smartphone offers outstanding battery life and an intelligent AI camera that captures stunning photos.."

print("Generated Review:\n")
print(generated_text)

# Sentiment analysis
sia = SentimentIntensityAnalyzer()
sentiment = sia.polarity_scores(generated_text)

print("\nSentiment Analysis:")
print(sentiment)

# Insight generation
if sentiment['compound'] > 0:
    print("\nInsight: The review is positive and suitable for marketing promotion.")
else:
    print("\nInsight: The review tone is neutral or negative.")
```

Generated Review:

This poor smartphone offers outstanding battery life and an intelligent AI camera that captures stunning photos..

Sentiment Analysis:
{'neg': 0.126, 'neu': 0.486, 'pos': 0.389, 'compound': 0.7579}

Insight: The review is positive and suitable for marketing promotion.
[nltk_data] Downloading package vader_lexicon to /root/nltk_data...

Result:

The code successfully establishes a concurrent connection. By using a unified class, you eliminate the need to manage different data formats
(JSON structures) manually, as the script flattens them into a standard string format.